

COMP7710

Programming Fundamentals

This week's lecture

Abstract Data Types (ADTs) and the Java Collections Framework

Course Convenor

Bernardo Pereira Nunes

Guest lecturer

Alberto F. Martin

Source: Core Java, Volume I: Fundamentals, 14th Edition
Chapter 9: Collections



Australian
National
University

Agenda

- > Intro to Abstract Data Types (ADTs)
- > The Java Collections Framework (i.e., ADTs in Java)
 - >> **Collection** and **Iterator** interfaces
 - >> Overview of main interfaces in the Collections Framework
 - >> Concrete class implementations of the **List** and **Queue** interfaces
 - >> Concrete class implementations of the **Set** and **Map** interfaces
 - >> Generic algorithms



Intro to Abstract Data Types (ADTs)

- > **Abstract Data Types** (ADTs) are mathematical models for data types
- > They are defined in terms of **behaviour** (*semantics*) from the user's perspective, **NOT** in terms of implementation (i.e., the specific details)
- > (This is why they are abstract, i.e., not concrete)
- > ADTs specify (1) possible values the data type may have (but NOT how they are laid out/stored in memory), (2) a set of operations that a user may perform on the data type (but NOT how they are implemented), and (3) behaviour (*semantics*) of such operations
- > ADTs are a **CORNERSTONE** for code reusability and modularity (as they enable code that relies on abstractions rather than concretions/specifics)
- > Examples of ADTs: **List, Queue, Deque, Set, Map, Tree, Graph, ...**
- > Typically, different programming languages offer different mechanisms to specify ADTs and their implementations



ADT example: List ADT

- > Let us illustrate the concept of ADTs with an example: the list ADT
- > For the purpose of the example, a list ADT represents a finite sequence of elements of the same type (e.g., list of strings, list of dates, list of employees, etc.)
- > Defining properties of the List ADT:
 - >> duplicate elements are allowed
 - >> order of elements is preserved (a unique index can be assigned to each element)
- > Some examples of list operations: *(there might be many more!)*
 - >> Add a new element to the end of the list
 - >> Remove first occurrence of an element from the list given the element
 - >> Get the element of the list at the specified index
 - >> Replace the element of the list at the specified index by the given element

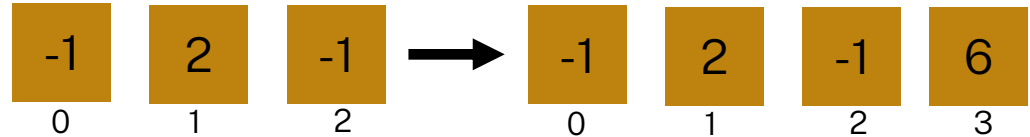


ADT example: List of Integers

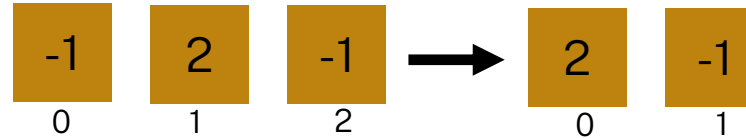
Example operation

Behaviour (semantics)

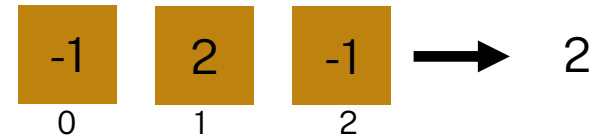
Add element 6



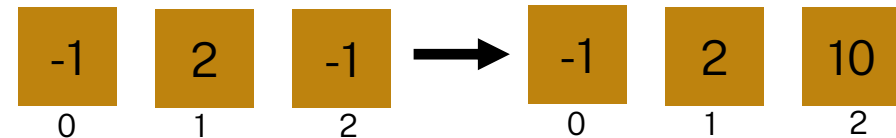
Remove element -1



Get element at index 1



Set index 2 to 10

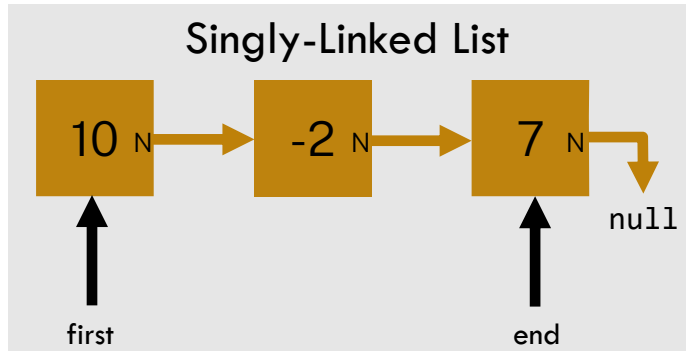


ADT implementations

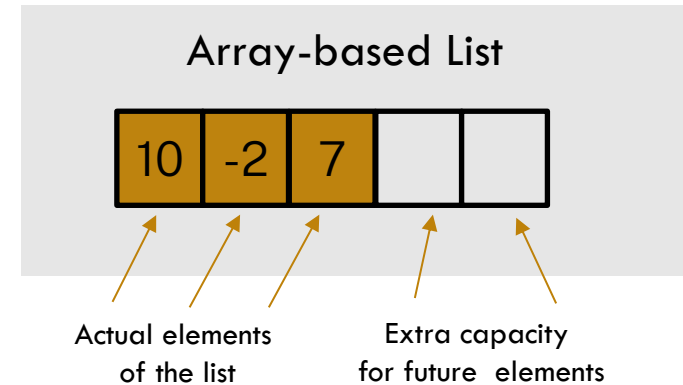
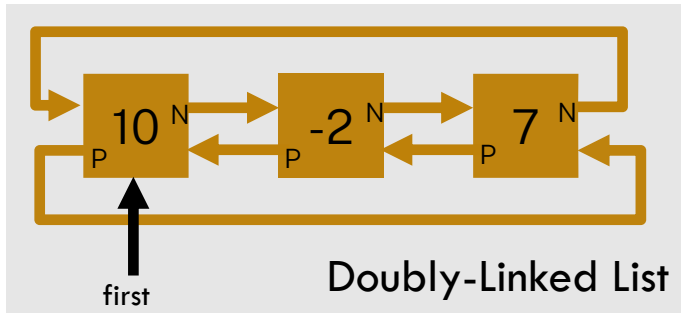
- > ADTs can typically be **realized** (implemented) in different ways. A realization of an ADT is called an ADT implementation
- > Any ADT implementation **MUST** adhere to the specified behaviour of the ADT and its operations (following the Liskov Substitution Principle)
- > However, implementations have the freedom to internally store/lay out in memory the elements differently, and thus to implement the ADT operations accordingly to the chosen internal data storage layout
- > Each ADT implementation typically leads to a different trade-off/balance among memory consumption and the computational cost of its operations
- > Which ADT implementation will be best for a particular algorithm will depend on the particular features of the algorithm at hand (this is why several implementations of an ADT exist)



Three examples of List ADT implementations



In Linked List implementations, the orange-colored boxes storing the list elements are called **nodes** and the arrows connecting the boxes are called **links**.
N stands for Next, and **P** for Previous.



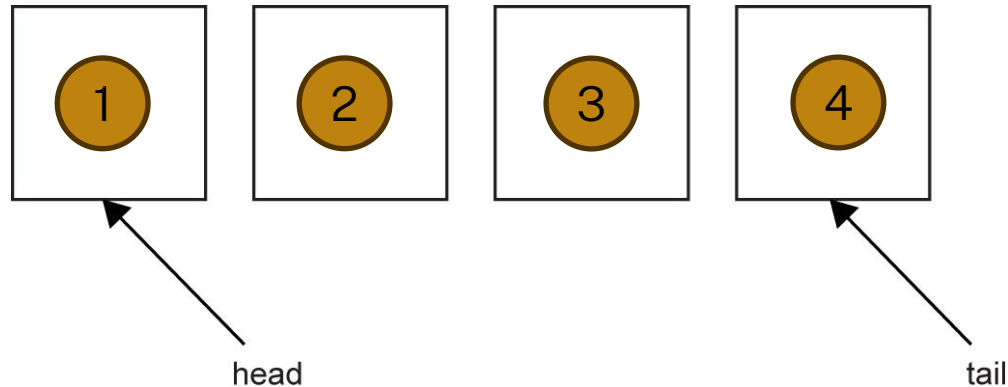
The Java Collections Framework (ADTs in Java)

- > Java offers different programming constructs that can be used to express ADTs/their implementations
- > Among these, a natural choice is to use **parameterized interfaces** to specify ADTs and **parameterized classes** implementing such interfaces for ADT implementations
- > Parameterized interfaces and classes are also known as **generic**. They are denoted by angle brackets + a placeholder type parameter, say, E, in their name, e.g., List<E> or ArrayList<E>
- > This is indeed the approach followed by the **Java collections framework** which, from Java 1.2, offers relatively small (compared to, e.g., C++) and easy-to-learn hierarchies of parameterized interfaces and class implementations, along with generic algorithms written in terms of such interfaces
- > Although we could define our own interfaces, it is a good practice to use those in the Java collections framework (so that we do not reinvent the wheel, allowing our class implementations to reuse generic code in the Java collections framework and beyond)
- > In the rest of the lecture, we will explore the basics of the Java collections framework, how to put it to work, and discuss on some of the design decisions.
- > **Note:** more details regarding parameterized definitions (i.e., generics) in the particular context of interfaces, classes and subtyping will be covered later in the course



Main principle of Java Collections illustrated by example (Queue)

- > As mentioned earlier, the Java Collections Framework **separates interfaces and implementations**
- > Let us illustrate the concept with another very common data structure in computing, the **Queue**
- > A Queue interface specifies that elements can be added at the tail end of the queue, and removed at the head of the queue. One may also ask how many elements there are in the Queue
- > A Queue preserves a so-called **FIFO (First-In First-Out)** order



Main principle of Java Collections illustrated by example (Queue)

> A simplified version of the Queue interface may look like:

```
public interface Queue<E> {  
    void add(E e); // Adds an element to the tail end of the queue  
    E remove();   // Removes and returns element at the head of the queue  
    int size();   // Returns how many elements there are in the queue  
}
```

Generic type parameter E of the interface

> As expected, the interface does not expose details on how the queue is implemented nor constructed

> Two possible class implementations (realizations) of the Queue interface in the Java Collections framework are the `ArrayDeque<E>` and `LinkedList<E>` classes

> The former is based on the concept of a “circular array” while the latter uses a Linked-List (next slide)



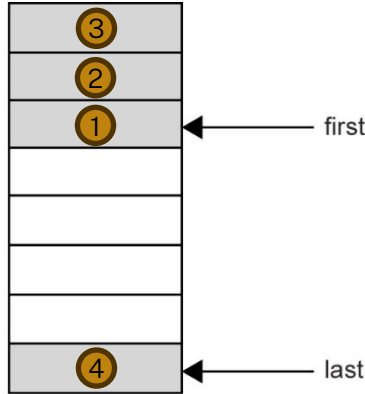
Main principle of Java Collections illustrated by example (Queue)

Generic type parameter **E** of the class

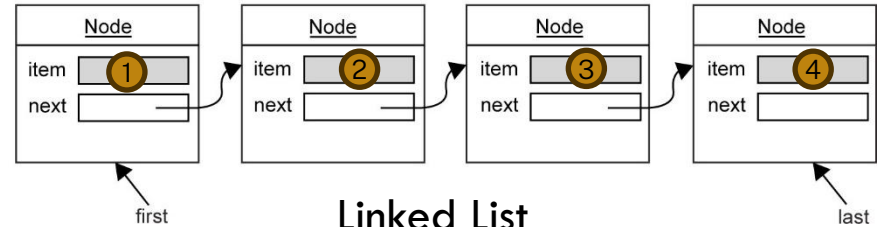
Generic type parameter **E** of the interface

```
public class ArrayDeque<E> implements Queue<E> {  
    private int first, last;  
    private E[] elements;  
    public ArrayDeque(int numElements) { ... }  
    public void add(E e) { ... }  
    public E remove() { ... }  
    public int size() { ... }  
    ...  
}
```

Circular
Array



```
public class LinkedList<E> implements Queue<E> {  
    private Node<E> first, Node<E> last;  
    public LinkedList() { ... }  
    public void add(E e) { ... }  
    public E remove() { ... }  
    public int size() { ... }  
    ...  
}
```



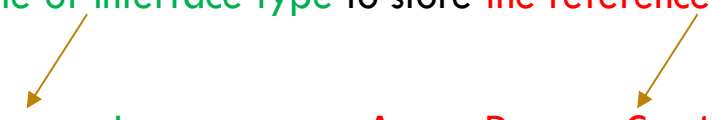
Linked List



Main principle of Java Collections illustrated by example (Queue)

- > When we use a Queue in our program, we do not actually need to know which implementation of the interface was used (that's the main point of separating interface and implementations)
- > Therefore, it is best practice in general to use the concrete class only when you construct the collection object (i.e., queue in our example)
- > To do so, we use a variable of interface type to store the reference to the class implementation:

```
% Queue<Customer> expressLane = new ArrayDeque<Customer>(100);  
% expressLane.add(new Customer("Harry"));
```



- > If we follow this best practice, and we change our mind about the class implementation, we only need to change the program in one place, i.e., the call to the implementation class constructor
- > For example, if we decided that a Linked List is a better choice, we can change the code above as:

```
% Queue<Customer> expressLane = new LinkedList<Customer>();  
% expressLane.add(new Customer("Harry"));
```



Main principle of Java Collections illustrated by example (Queue)

- > We also want to use variables of interface type (instead of concrete class type) because typically the class implementations include methods beyond that in the interface
- > For example, ArrayDeque and LinkedList have methods that are totally unrelated to the Queue interface, but to comply with other interfaces (e.g., LinkedList also implements the List interface)
- > Accidentally using one of these methods from a variable of interface type Queue, immediately results in a compile-time error
- > Which implementation is best in practice?
 - >> A circular array (i.e., ArrayDeque) is more memory-efficient
 - >> However, it has finite capacity, and needs to be re-allocated whenever it fills up
 - >> Although the answer depends on the specific use-case, evidence shows that the ArrayDeque generally outperforms a LinkedList because the former has less impact on the garbage collector
- > Using the best practices described above, when the ArrayDeque appeared in Java 6, programmers were able to enjoy its performance benefits by just changing a single line of code (which one???)



Collection and Iterator interfaces

- > One of the most fundamental interfaces in the Java collections Framework is the **Collection** interface
- > It conceptually represents a collection of objects of a given type not necessarily arranged in a predefined order (some sub-interfaces may impose an order of the elements, e.g., List or Queue)
- > The Collection interface has two fundamental methods:

```
public interface Collection<E> extends ...  
{  
    boolean add(E e);  
    Iterator<E> iterator();  
    ...  
}
```

>> The add method adds an element to the collection

>> It returns true if adding the element actually changes the collection, and false otherwise

>> For example, if one tries to add an element to a set collection, and the element already exists in the set, then the add method returns false, as the set rejects duplicates

>> The iterator method returns an object belonging to a class that implements the **Iterator** interface (next slide)

>> With that object one may visit the elements of the collection one by one



The Iterator interface

> The iterator interface has four methods:

```
public interface Iterator<E>
{
    boolean hasNext();
    E next();
    default void remove();
    default void forEachRemaining(Consumer<? super E> action);
}
```

> One may visit all the elements in the collection one by one by calling repeatedly the next method (typically in a loop)

> Care should be taken when calling the next method, as it throws a NoSuchElementException if the iterator has already reached the end of the collection

> To avoid such a situation, one can double check that the iterator has not yet reached the end of the collection by calling hasNext()

> hasNext() returns true if there is still at least one element to be visited



The Iterator interface

> The general recipe for working with iterators first involves requesting an iterator and then to keep calling the next method till hasNext returns false:

```
% Collection<String> coll = ...
% Iterator<String> iter = coll.iterator()
% while (iter.hasNext()) {
    String element = iter.next();
    do something with element;
}
```

The order in which the elements are visited depends on the particular collection type. For example, in a List, the elements are visited by increasing index. However, in a Set, the order is undefined, that is, any order is possible as far as all elements are encountered during the course of the iteration

> Alternatively, one may write this process more concisely with a “for each” loop:

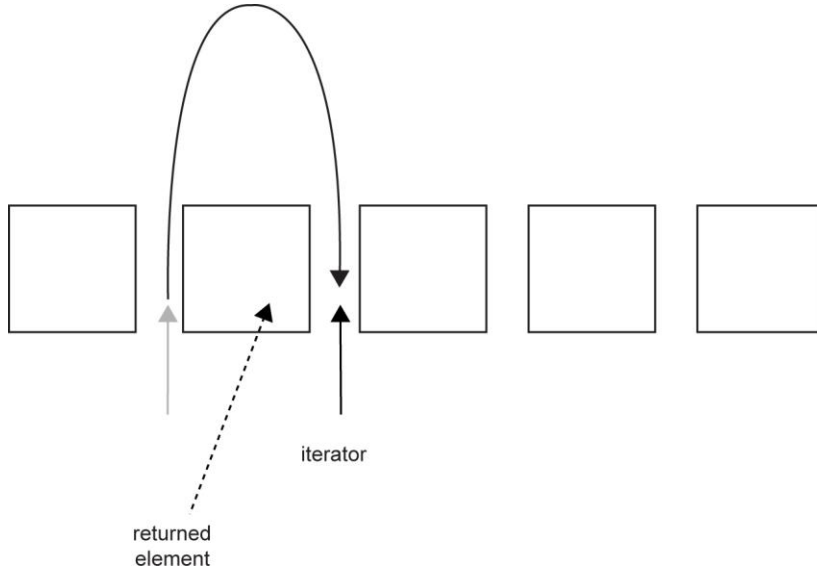
```
% for (String element : coll ) {
    do something with element;
}
```

> The “for each” loop works with any object that implements the **Iterable** interface. The Collection interface extends the **Iterable** interface. Thus, one can use the “for each” loop with any class implementation of the Collection interface



The Iterator interface

- > When working with iterators, it significantly helps to think about them as being *between elements*
- > Initially, the iterator is located right before the first element
- > When one calls the next method, the iterator **jumps over the next element**, and it returns a reference to the element it just passed



This in contrast to other OO languages, such as C++, for which iterators actually point to the elements themselves, and not in between elements. Besides, in such iterators one may retrieve the element without necessarily advancing the iterator forward



The Iterator interface

- > Another way to visit the elements in a collection is through the `Collection.forEach` and `Iterator.forEachRemaining` methods
- > These methods receive a **lambda expression** that consumes an element, once at a time
- > The lambda expression is invoked either for all elements in the collection or the remaining elements that the iterator is yet to visit, respectively:

```
% coll.forEach(element->do something with element)
% iterator.forEachRemaining(element->do something with element)
```

TO-THINK: Imagine that you have an object belonging to a class that implements the `Collection<Integer>` interface, say `ArrayList<Integer>`. How would you print on screen all the numbers in the collection using the `forEach` method + a lambda expression? How would you do so with an iterator?



The Iterator interface

- > The `remove` method of Iterator interface removes the element that was returned by last call to `next`
- > The rationale behind this design decision is that in order to be able to determine whether an element is to be removed from the collection, one first needs to inspect the element before taking the decision
- > However, that implies that if one wants to eliminate an element in a particular position, then one must still skip past the element from the first element
- > For example, here is a code of how one would remove the first element in a collection:

```
% Iterator<String> iter = coll.iterator()  
% if (iter.hasNext()) {  
    iter.next();  
    iter.remove();  
}
```



TO-THINK: Why the if statement is needed?

- > It is **illegal** to call the `remove` method without a preceding `next` call. If we do an `IllegalStateException` will be thrown
- > Besides, if one wants to eliminate two adjacent elements, it is illegal to call `remove` twice. One must call `next` before each call to `remove`



Generic Collection Utility Methods

> The Collection and Iterator interfaces are generic, meaning that you can write utility methods that operate on any kind of Collection (*even for those that do not yet exist!*)

> For example, a generic static method that tests whether an arbitrary Collection contains a given element may look like:

```
import java.util.Collection;
import java.util.ArrayList;
public class CollectionUtilityMethods {
    public static <E> boolean contains(Collection<E> coll, Object element) {
        for (E coll_element: coll) {
            if (coll_element != null && coll_element.equals(element)) {
                return true;
            }
        }
        return false;
    }
    ...
}
```

```
% Collection<String> names = new ArrayList<String>();
% names.add("Henry")
% names.add("Joe")
% names.add("Lucke")
% CollectionUtilityMethods.contains(names, "Joe"); // returns true
```

This method can be used with objects of any class that implements the Collection interface. If, say, in two years time after writing this method, we need to develop another class implementing Collection, **we can re-use the method as-is without touching a single line of code**



Generic Collection Utility Methods

> The designers of the Java Collections Framework decided that some of these utility methods are so useful that the framework should make them available. This way, API users don't have to keep reinventing the wheel. The contains method in the previous slide is an example of such methods

> Indeed, the Collection<E> interface declares a few utility methods that all classes adhering to the interface must implement:

```
int size()
boolean isEmpty()
boolean contains(Object obj)
boolean containsAll(Collection<?> c)
boolean add(E obj)
boolean addAll(Collection<? extends E> from)
boolean remove(Object obj)
boolean removeAll(Collection<?> c)
boolean retainAll(Collection<?> c)
void clear()
Object[] toArray()
<T> T[] toArray(T[] a)
default <T> T[]
    toArray(IntFunction<T[]> generator)
```

For backwards compatibility, the input argument to the contains and remove methods is of type Object, instead of generic type E. That implies, that one may try, for, example, to **remove an String** from, say, a collection of Integers, without the compiler throwing any compilation error

For the same reason, the input collection to containsAll, removeAll, and retainAll, is declared as Collection<?> instead of Collection<? extends E>. “?” denotes the so-called **unlimited wild-card**. Essentially it means collections where the elements might be of any type, potentially different from E (but with all collection elements of that same type)

The last two toArray methods in a collection let us generate an array of elements of a specific type T from the Collection. Note that T is a type parameter tight to these methods, and potentially different to the interface generic type (although in most cases it will be equal to E, e.g., when transforming a collection of Integers into an array of Integers).

The last toArray method accepts a lambda expression. Valid lambda expressions for this method can be generated with the notation T[]::new, as we saw in the lambdas lecture



Generic Collection Utility Methods

> To make the life easier to implementors of the `Collection<E>` interface, Java supplies **an abstract class**, `AbstractCollection<E>`, that leaves the fundamental methods `size` and `iterator` abstract, but implements most of the methods of `Collection<E>` in terms of these two

```
public abstract class AbstractCollection<E> implements Collection<E> {
    ...
    public abstract Iterator<E> iterator();
    public abstract int size();
    public boolean isEmpty() {
        return size() == 0;
    }
    public boolean contains(Object o) {
        Iterator<E> it = iterator();
        while (it.hasNext()) {
            E element = it.next();
            if (element != null && element.equals(o)) return true;
        }
        return false;
    }
    ...
}
```

>> A concrete class implementation can now extend the `AbstractCollection<E>` class, and **MUST** implement the `size` and `iterator` methods. Mutable collections also need the `add` method. The other methods are taken care of by `AbstractCollection`. However, an implementor may still override some of these default implemented methods if there is more efficient way to implement them

>> Another alternative would have been to use default interface methods, but these sort of methods did not actually exist when the `Collections` framework was designed. Still, some default methods have been added to the `Collections` interface afterwards, such, as, e.g., the last `toArray` method in the previous slide



java.util.Collection<E> interface (Section 9.1, Core Java Vol I book) Part 1/2

Method Signature	Description
<code>Iterator<E> iterator()</code>	returns an iterator that can be used to visit the elements in the collection
<code>int size()</code>	returns the number of elements currently stored in the collection
<code>boolean isEmpty()</code>	returns true if this collection contains no elements
<code>boolean contains(Object obj)</code>	returns true if this collection contains an object equal to <code>obj</code>
<code>boolean containsAll(Collection<?> other)</code>	returns true if this collection contains all elements in the other collection
<code>boolean add(E element)</code>	attempts to add an element to the collection. Returns true if the collection changed as a result of this call
<code>boolean addAll(Collection<? extends E> other)</code>	adds all elements from the other collection to this collection. Returns true if the collection changed as a result of this call



java.util.Collection<E> interface (Section 9.1, Core Java Vol I book) Part 2/2

Method Signature	Description
<code>boolean remove(Object obj)</code>	attempts to remove an object equal to <code>obj</code> from this collection. Returns <code>true</code> if a matching object was removed
<code>boolean removeAll(Collection<?> other)</code>	removes from this collection all elements from the other collection. Returns <code>true</code> if the collection changed as a result of this call
<code>default boolean removeIf(Predicate<? super E> filter)</code>	removes all elements for which <code>filter</code> returns <code>true</code> . Returns <code>true</code> if the collection changed as a result of this call
<code>void clear()</code>	removes all elements from this collection
<code>boolean retainAll(Collection<?> other)</code>	removes all elements from this collection that do not equal one of the elements in the other collection. Returns <code>true</code> if the collection changed as a result of this call
<code>Object[] toArray()</code>	returns an array of the objects in the collection
<code>default <T> T[] toArray(IntFunction<T[]> generator)</code>	returns an array of the objects in the collection. The array is constructed with the <code>generator</code> , which is typically a constructor expression <code>T[] : new</code>

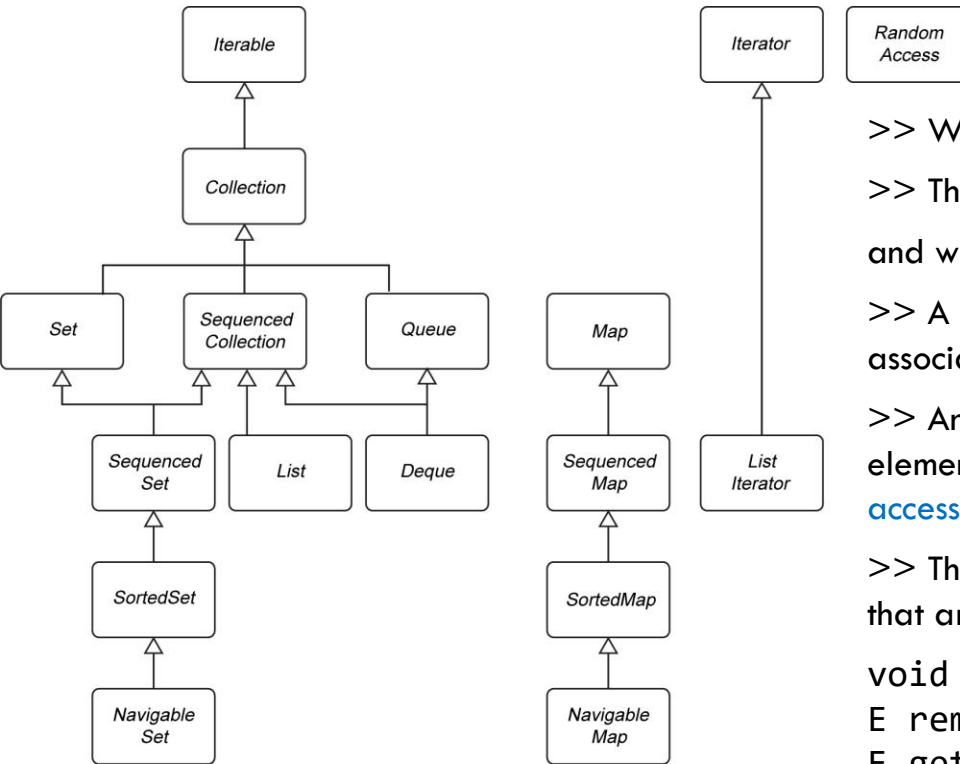


java.util.Iterator<E> interface (Section 9.1, Core Java Volume I book)

Method Signature	Description
<code>boolean hasNext()</code>	returns true if there is another element to visit
<code>E next()</code>	returns the next object to visit. Throws a <code>NoSuchElementException</code> if the end of the collection has been reached
<code>void remove()</code>	removes the last visited object. This method must immediately follow an element visit. If the collection has been modified since the last element visit, this method throws an <code>IllegalStateException</code> . Iterators are not required to support this operation. The default implementation throws an <code>UnsupportedOperationException</code>
<code>default void forEachRemaining(Consumer<? super E> action)</code>	visits elements and passes them to the given action until no elements remain or the action throws an exception.



Overview of main interfaces in the Collections Framework



>> We have already visited `Iterable`, `Collection`, `Queue`, and `Iterator`

>> The `Map` interface represents a map among keys and values and will be covered later in the lecture

>> A `List` represents an **ordered collection**. Elements have an associated index, and are added in a particular position

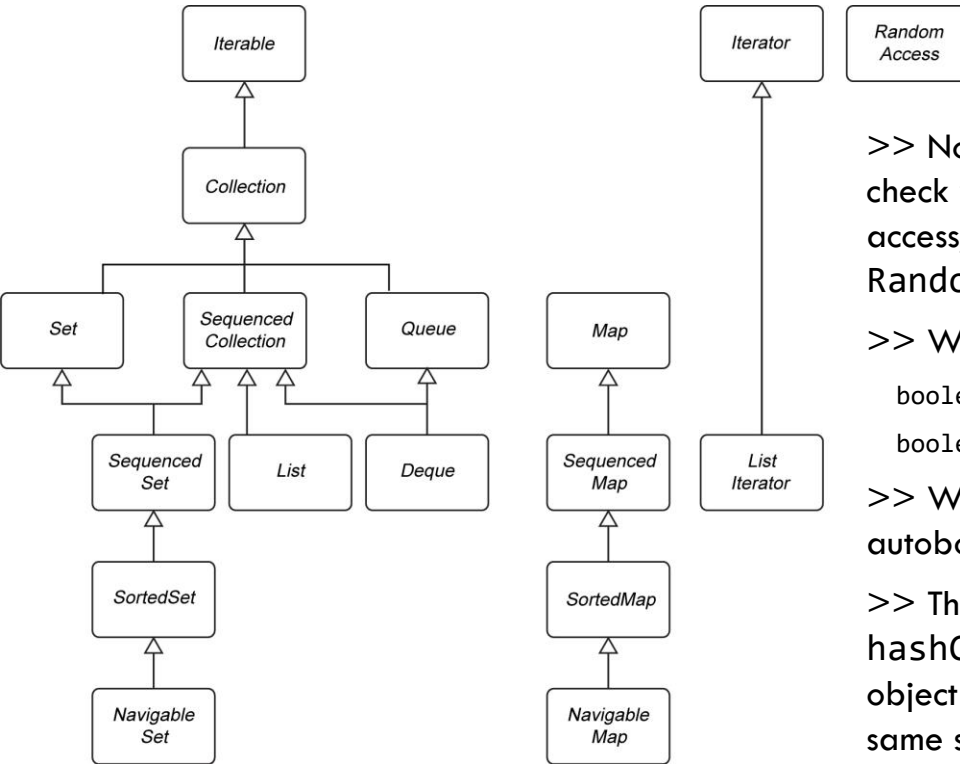
>> An element in a `List` can be accessed in two main ways: by its element index, or by an iterator. The former is referred to as **random access** because elements can be accessed in any order

>> The `List<E>` interface offers several methods for **random access** that are not in the `Collection<E>` interface (**why not???**):

```
void add(int index, E element)
E remove(int index)
E get(int index)
E set(int index, E element)
```



Overview of main interfaces in the Collections Framework



>> Not all `List` implementations support efficient random access. To check whether a `List` implementation supports efficient random access, one can check if such implementation is an instance of the `RandomAccess` interface

>> With `List<Integer>` there are two remove methods (!!!):

```
boolean remove(int index) // Removes element with the given index
```

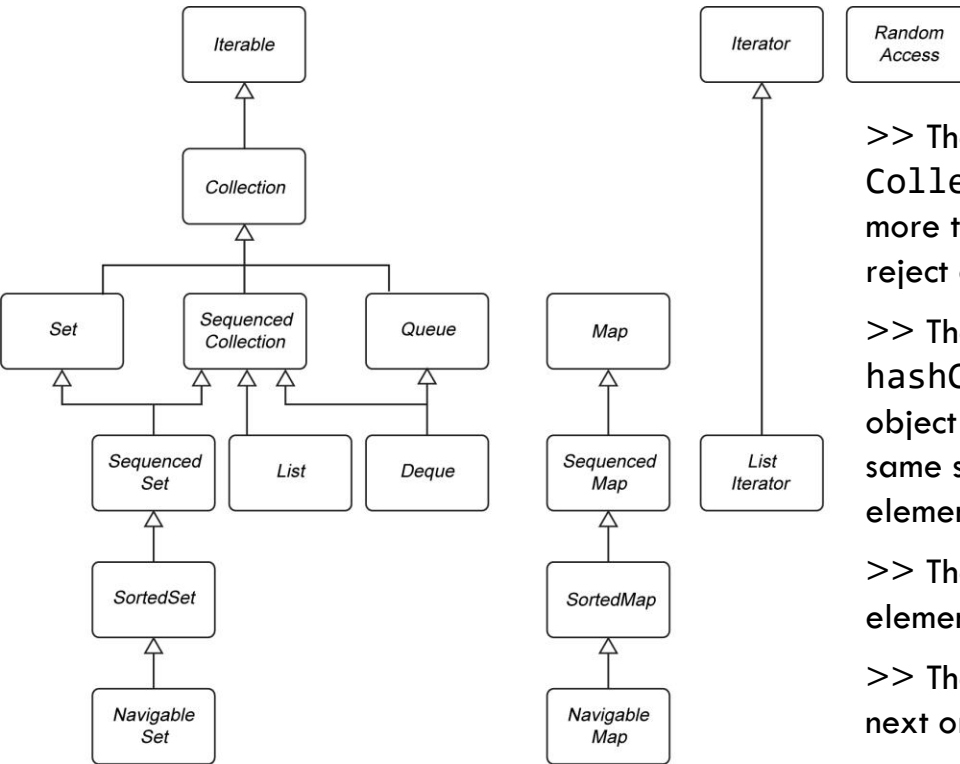
```
boolean remove(Integer o) // Removes element equal to o
```

>> When calling `remove` with an `int`, the first method is called. No autoboxing is considered, and there is no warning (DANGER)

>> The `List` interface fixes the semantics of the `equals` and `hashCode` methods. For a list to be equal to another object, the other object has to be a `List` of some kind. Besides, both lists must have the same size, and corresponding elements must be equal



Overview of main interfaces in the Collections Framework



>> The Set interface has the same instance methods as the Collection interface, but the behaviour of some of its methods is more tightly defined. For example, the add method of a Set should reject duplicates and return false

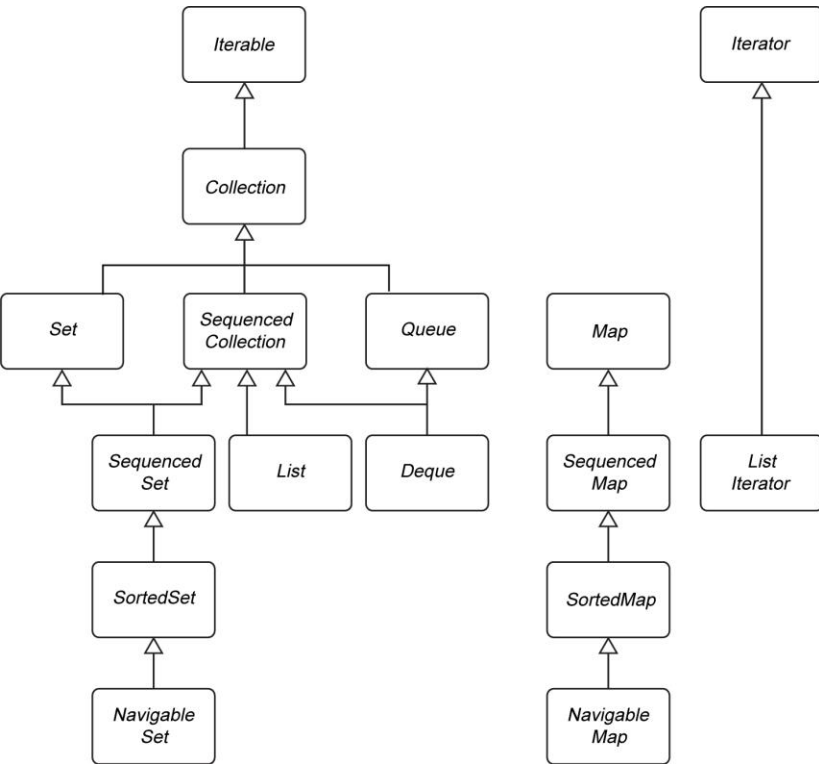
>> The Set interface fixes the semantics of the equals and hashCode methods. For a set to be equal to another object, the other object has to be a Set of some kind. Besides, both sets must have the same size, and the same elements to be equal. Two sets with the same elements thus must have the same hash code.

>> The SortedSet exposes the Comparator object used to sort the elements of the set

>> The NavigableSet contains additional methods for finding the next or previous element in sorted sets



Overview of main interfaces in the Collections Framework



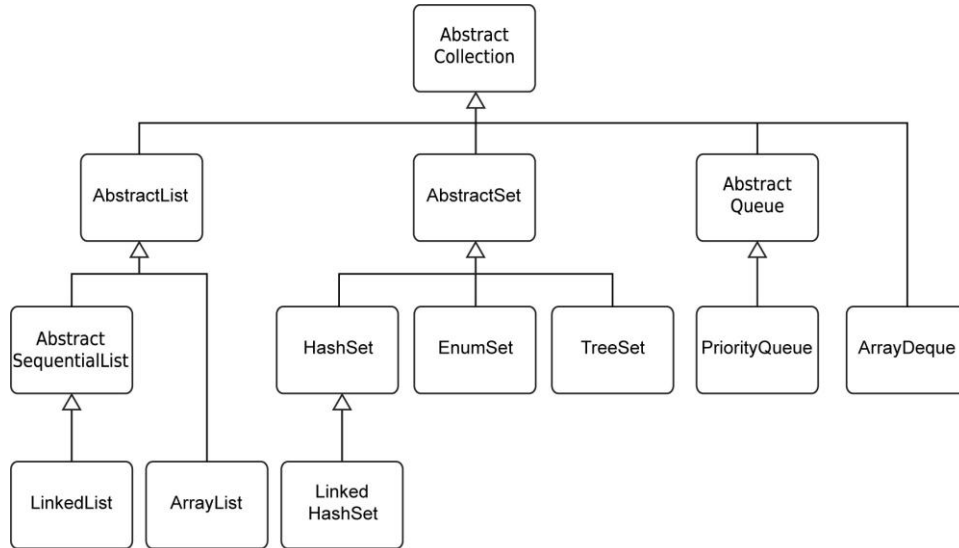
>> Java 21 introduced the `SequenceCollection<E>` interface, that provides access to the first and last elements of a collection, and reverse traversal:

```
E getFirst(); E getLast()
void addFirst(E e); void addLast(E e)
E removeFirst(); E removeLast()
SequenceCollection<E> reversed()
```

>> The `SequenceSet<E>` interface sharpens the return type of the `reversed` method to `SequenceSet<E>`



Concrete collection implementations

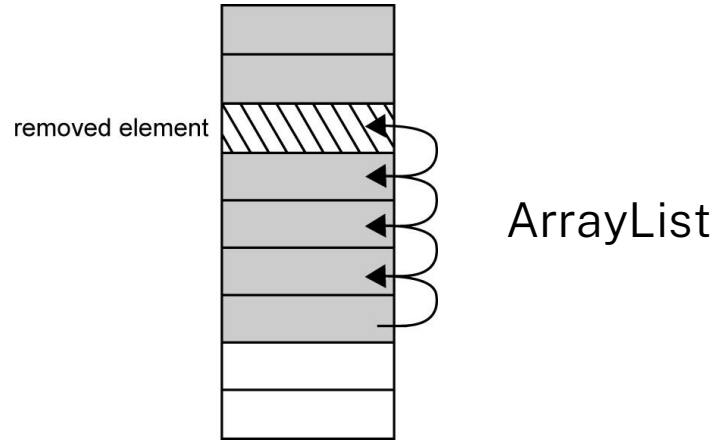


Collection Type	Description
ArrayList	An indexed sequence that grows and shrinks dynamically
LinkedList	An ordered sequence that allows efficient insertion and removal at any location
ArrayDeque	A double-ended queue that is implemented as a circular array
PriorityQueue	A collection that allows efficient removal of the smallest element
HashSet	An unordered collection that rejects duplicates, and allows for efficient lookup of elements
TreeSet	A sorted set
EnumSet	A set of enumerated type values
LinkedHashSet	A set that remembers the order in which the elements were inserted



The main drawback of Arrays and ArrayLists

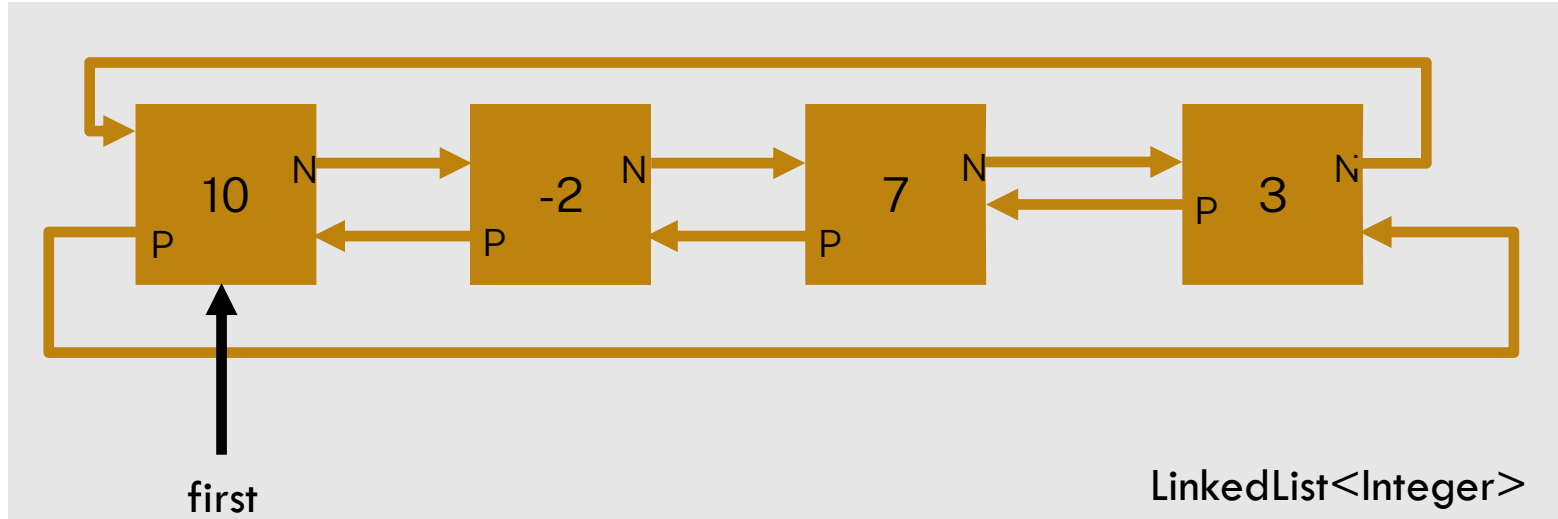
- > We have been using Arrays and their dynamic variant ArrayLists for a while in this course
- > Arrays and ArrayLists suffer from **a major performance drawback**: removing an element is generally expensive since all elements beyond the removed one must be moved towards the beginning



- > The same applies when inserting elements in the array, although in this case the elements beyond the inserted one must be moved towards the end of the array

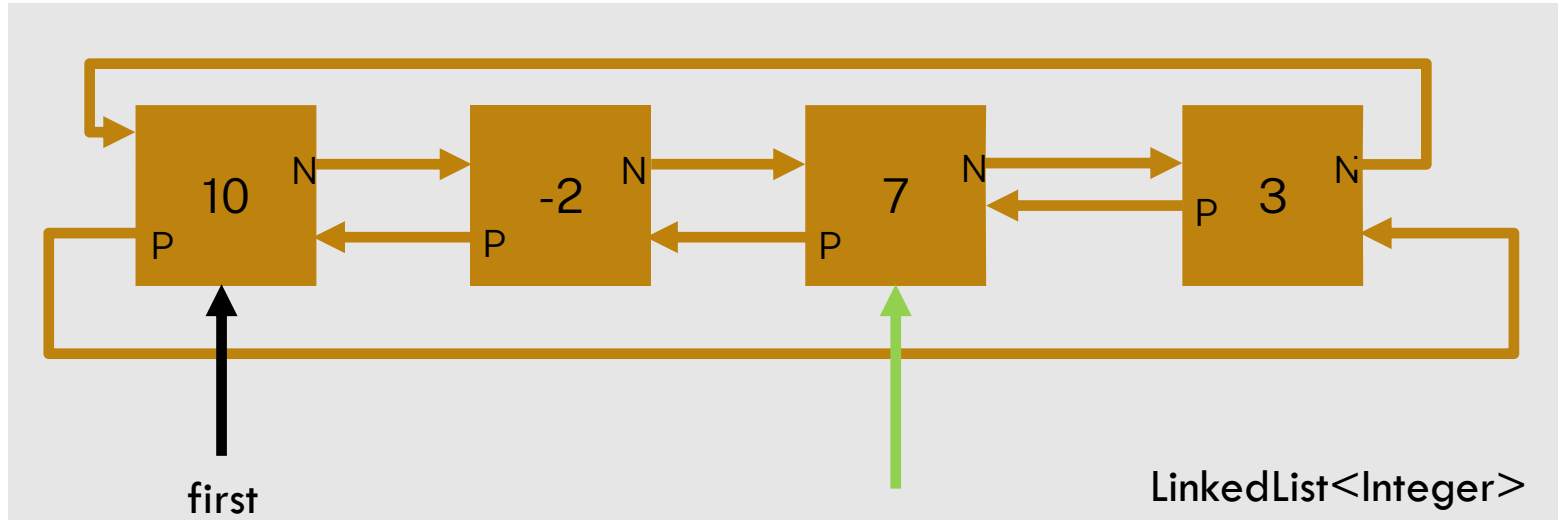
Linked Lists

- > Another well-known data structure, the **linked list**, solves this problem
- > A linked list stores each object in a separate node.
- > Each node holds a reference to the next node (N in the figure)
- > In the Java Collections Framework, linked lists are **doubly linked**: each node also stores a reference to its predecessor (P in the figure)

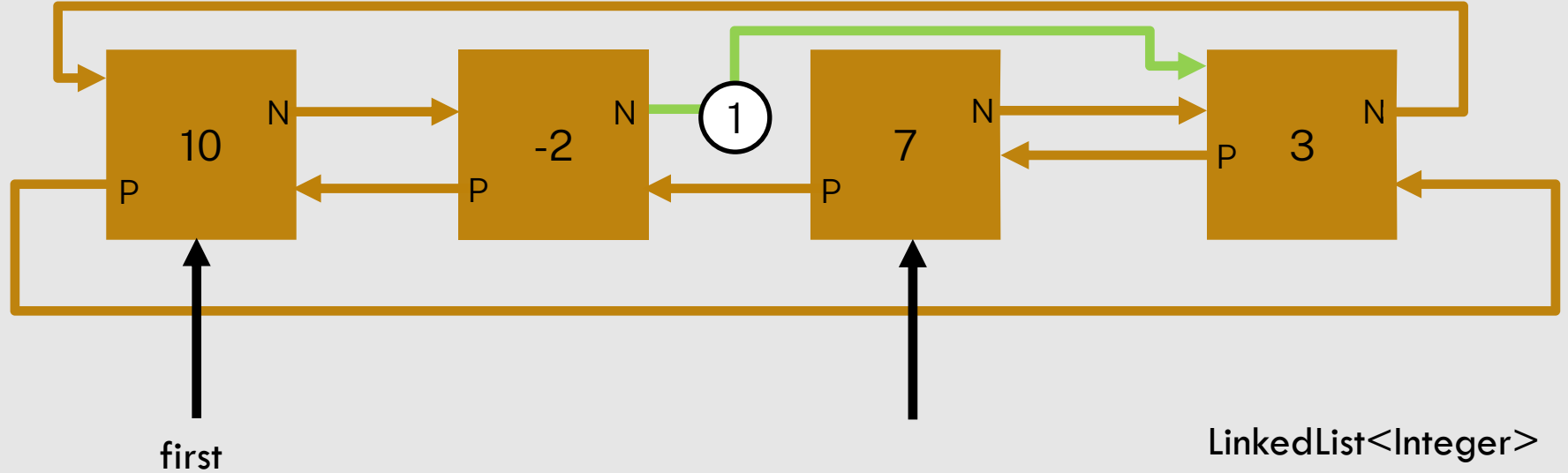


Linked Lists

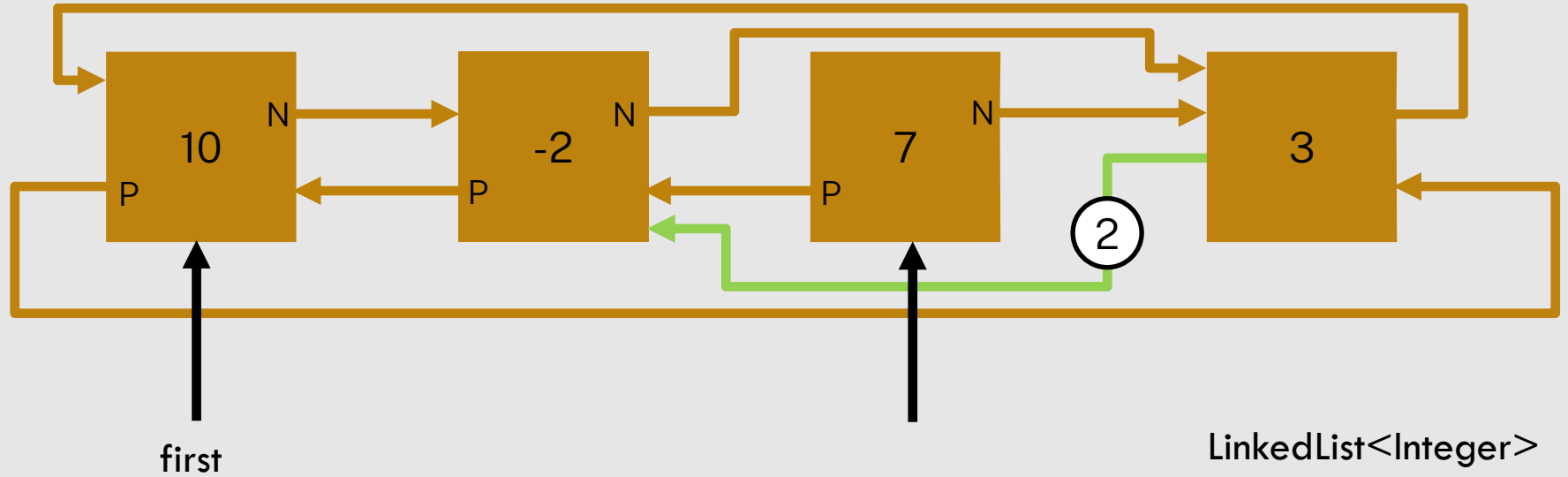
- > **Removing an element from the middle of a linked list is an inexpensive operation** – only the links around the element to be removed need to be updated
- > How would you remove element 7 in the linked list below? Assume that you have a reference to the node that stores the element 7 (i.e., green arrow)



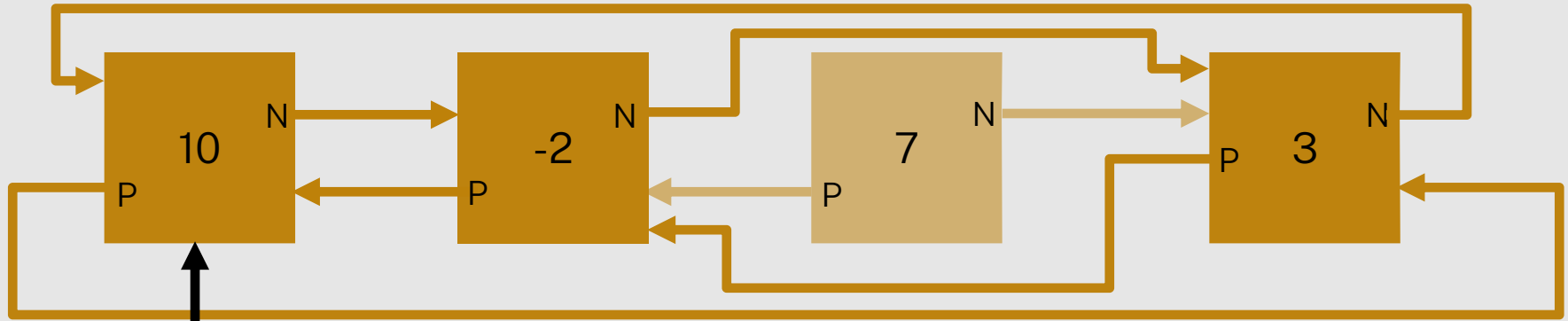
Linked Lists



Linked Lists



Linked Lists



After this process, how many references are pointing to the node that stores the element 7?

`LinkedList<Integer>`



Linked Lists

> The class `LinkedList<E>` handles these references under the hood when removing or adding elements in the linked list, saving programmers from this burden

> For example, the following code adds three elements to the list, and then removes the second one:

```
var staff = new LinkedList<String>();  
staff.add("Amy"); staff.add("Bob"); staff.add("Carl");  
Iterator<String> iter = staff.iterator();  
String first = iter.next(); // visit first element (Amy)  
String second = iter.next(); // visit second element (Bob)  
iter.remove(); // remove last visited element (Bob)
```

> The `LinkedList.add(...)` method adds the object at the end of the list. However, in many different scenarios, we may want to add objects somewhere in the middle of the list

> To this end, lists provide a specialized iterator to add elements to the list. Using iterators to add elements makes sense only for collections that have a natural ordering, as lists. For example, the iterator for sets does not allow to add elements, as sets do not have a natural ordering



Lists Iterators

> The Collections Framework supplies a subinterface `ListIterator<E>` that contains an `add` method:

```
interface ListIterator<E> extends Iterator<E> {  
    void add(E element); ...  
}
```

> Unlike `Collections.add`, this method does not return a boolean – it is assumed that the `add` operation always modifies the list

> If we call the `ListIterator.add` method multiple times, the elements are added in the order in which we supplied them, **before the current iterator position**

> Besides, the `ListIterator` interface has two methods that we can use for traversing the list backwards:

```
boolean hasPrevious(); E previous();
```

> Like the `next` method, the `previous` method returns the object that iterator skipped over. Right after a call to `previous`, the `remove` method of the iterator eliminates the element to the right of the iterator position

> The `listIterator()` method of the `LinkedList` class returns an object of a class that implements the `ListIterator<E>` interface



Lists Iterators (examples)

The pipe character “|” denotes the current position of the list iterator

it = l.listIterator()

it.previous()

① A B C D E F → | A B C D E F ⑦ A G H | C D E F → A G | H C D E F

it.next()

it.remove()

② | A B C D E F → A | B C D E F ⑧ A G | H C D E F → A G | C D E F

it.add(G)

③ A | B C D E F → A G | B C D E F

it.add(H)

④ A G | B C D E F → A G H | B C D E F

it.next()

⑤ A G H | B C D E F → A G H B | C D E F

it.remove()

⑥ A G H B | C D E F → A G H | C D E F

Note how the add method just depends on the current position of the list iterator. **HOWEVER**, the remove method depends **BOTH** on the current position and state of the list iterator (i.e., whether the last operation with the iterator was next or previous)



Linked Lists

> Linked Lists do **NOT** support **fast random access**: if we want to obtain the n-th element, we must start at the beginning and skip past the first n-1 elements

> Nevertheless, the LinkedList class supplies a get method that lets you access a particular element:

```
LinkedList<String> list = ...
```

```
String n_th_element = list.get(n)
```

> This method is **NOT** efficient. If we find ourselves using it, we are probably using a wrong data structure for our problem at hand

> For example, the code below is VERY inefficient:

```
for (int i = 0; i < list.size(); i++)  
    do something with list.get(i);
```

> Each time the loop looks for a new element, the get method starts again the search from the beginning of the list

> This problem does not actually appear with ArrayLists, which provide fast random access to their elements, i.e., `ArrayList.get(n)` is VERY efficient



java.util.List<E> interface (Section 9.3, Core Java Volume I book)

Method Signature	Description
<code>ListIterator<E> listIterator()</code>	returns a list iterator for visiting the elements of the list
<code>ListIterator<E> listIterator(int index)</code>	returns a list iterator for visiting the elements of the list whose first call to <code>next</code> will return the element with the given index
<code>void add(int i, E element)</code>	adds an element at the specified position
<code>boolean addAll(int i, Collection<? extends E> elements)</code>	adds all elements from a collection to the specified position
<code>E remove(int i)</code>	removes the element at the specified position
<code>E get(int i)</code>	gets the element at the specified position
<code>E set(int i, E element)</code>	replaces the element at the specified position with a new element and returns the old element
<code>int indexOf(Object element)</code>	returns the position of the first occurrence of an element equal to the specified element, or <code>-1</code> if no matching element is found
<code>int lastIndexOf(Object element)</code>	returns the position of the last occurrence of an element equal to the specified element, or <code>-1</code> if no matching element is found



java.util.ListIterator<E> interface (Section 9.3, Core Java Volume I book)

Method Signature	Description
<code>void add(E newElement)</code>	adds an element before the current position.
<code>void set(E newElement)</code>	replaces the last element visited by <code>next</code> or <code>previous</code> with a new element. Throws an <code>IllegalStateException</code> if the list structure was modified since the last call to <code>next</code> or <code>previous</code> .
<code>boolean hasPrevious()</code>	returns <code>true</code> if there is another element to visit when iterating backwards through the list.
<code>E previous()</code>	returns the previous object. Throws a <code>NoSuchElementException</code> if the beginning of the list has been reached.
<code>int nextIndex()</code>	returns the index of the element that would be returned by the next call to <code>next</code> , or the size of the list if the iterator is past the last element.
<code>int previousIndex()</code>	returns the index of the element that would be returned by the next call to <code>previous</code> , or <code>-1</code> if the iterator is before the first element.



java.util.LinkedList<E> class (Section 9.3, Core Java Volume I book)

Method Signature	Description
<code>LinkedList()</code>	constructs an empty linked list.
<code>LinkedList(Collection<? extends E> elements)</code>	constructs a linked list and adds all elements from a collection.
<code>void addFirst(E element); void addLast(E element)</code>	add an element to the beginning or the end of the list.
<code>E getFirst(); E getLast()</code>	return the element at the beginning or the end of the list.
<code>E removeFirst(); E removeLast()</code>	remove and return the element at the beginning or the end of the list.



Take-away exercise

Write a **generic** class `DoublyLinkedList<E>`, parameterized by the type of the elements `E`, that implements a partial set of the methods in the `List<E>` interface, namely:

(1) **size**; (2) **isEmpty**; (3) **add** a new element; (4) **remove** an element; (5) **get**; (6) **set**; (7) **containsAll**; (8) **iterator**.

In order to get you started, the code supplied along with these slides includes the class definitions needed, and the implementation of (2), (3), and (8).

Your task is to implement (1), (4), (5), (6), and (7), and add some code to test these methods in the `DoublyLinkedListDemo.java` file.



Queues and Deques

- > As shown earlier, a **queue** lets us efficiently add an element after the last (**tail**) and remove the first (**head**). With a doubled-ended queue (**deque**) we can efficiently add/remove elements at both ends
- > In any of these two collections, adding or removing elements in the middle is **NOT** supported
- > Queues and Deques are provided by the `Queue<E>` and `Deque<E>` interfaces, respectively. The `Deque<E>` interface extends the `Queue<E>` interface
- > Both the `ArrayDeque<E>` and `LinkedList<E>` classes implement the `Deque<E>` interface (and thus the `Queue<E>` interface as well). **These collections can grow in size dynamically as needed**
- > In Java 21, the methods:

```
E getFirst(); E getLast()  
void addFirst(E e); void addLast(E e)  
E removeFirst(); E removeLast()  
SequencedCollection<E> reversed()
```

were moved from the `Deque<E>` interface to the `SequencedCollection<E>` interface, which, e.g., the `List<E>` interface also extends



java.util.Queue<E> interface (Section 9.3, Core Java Vol I book)

Method Signature	Description
<code>boolean add(E e);</code> <code>boolean offer(E e);</code>	add the given element after the last element of this queue and return true, provided the queue is not full. If the queue is full, the first method throws an <code>IllegalStateException</code> , whereas the second method returns false
<code>E remove();</code> <code>E poll();</code>	remove and return the first element of this queue, provided the queue is not empty. If the queue is empty, the first method throws a <code>NoSuchElementException</code> , whereas the second method returns null
<code>E element();</code> <code>E peek();</code>	return the element at the head of this queue without removing it, provided the queue is not empty. If the queue is empty, the first method throws a <code>NoSuchElementException</code> , whereas the second method returns null



java.util.SequencedCollection<E> interface

(Section 9.3, Core Java Vol I book)

Method Signature	Description
<code>void addFirst(E element);</code> <code>void addLast(E element);</code>	add the given element before the first or after the last element of this collection. If the collection is full, throw an <code>IllegalStateException</code>
<code>E removeFirst();</code> <code>E removeLast();</code>	remove and return the first or last element of this collection, provided the collection is not empty. If the collection is empty, throw a <code>NoSuchElementException</code>
<code>E getFirst();</code> <code>E getLast();</code>	return the first or last element of this collection without removing it, provided the collection is not empty. If the collection is empty, throw a <code>NoSuchElementException</code>



java.util.Deque<E> interface (Section 9.3, Core Java Vol I book)

Method Signature	Description
<code>boolean offerFirst(E element);</code> <code>boolean offerLast(E element);</code>	add the given element before the first or after the last element of this deque. If the deque is full, return false
<code>E pollFirst();</code> <code>E pollLast();</code>	remove and return the first or last element of this deque, provided the deque is not empty. If the deque is empty, return null
<code>E peekFirst();</code> <code>E peekLast();</code>	return the first or last element of this deque without removing it, provided the deque is not empty. If the deque is empty, return null

java.util.ArrayDeque<E> class (Section 9.3, Core Java book)

Method Signature	Description
<code>ArrayDeque()</code>	construct an unbounded deque with an initial capacity of 16
<code>ArrayDeque(int initialCapacity)</code>	construct an unbounded deque with the given initial capacity



Priority Queues (PriorityQueue<E>)

- > A priority queue allows element retrieval in sorted order after inserting them in arbitrary order
- > That is, whenever we call the `remove` method, we get the “smallest” element currently in the queue
- > HOWEVER, the priority queue does **NOT** actually sort the elements internally
- > Indeed, if we iterate over a priority queue, we may not necessarily get the elements in sorted order
- > Priority queues are implemented using a very relevant data structure in computing: the **heap**
- > A heap is a self-organizing binary tree in which the add and remove operations cause the smallest element to always gravitate to the root, without the need to keep the elements sorted
- > A priority queue must hold objects implementing the `Comparable<E>` interface
- > Alternatively, one may supply an object implementing the `Comparator<E>` interface to the constructor of the priority queue
- > A typical use case of priority queues is job scheduling (e.g. in supercomputers):
 - >> Each job has a priority
 - >> Jobs are added in random order
 - >> Whenever a new job can be started, the highest priority job is removed from the priority queue



Priority Queues (Example)

> The code below show priority queues in action:

```
import java.util.PriorityQueue; import java.time.LocalDate;

class PriorityQueueDemo {
    public static void main(String [] args) {
        var pq = new PriorityQueue<LocalDate>();
        pq.add(LocalDate.of(1906, 12, 9)); // G. Hopper
        pq.add(LocalDate.of(1815, 12, 10)); // A. Lovelace
        pq.add(LocalDate.of(1903, 12, 28)); // J. von Neumann
        pq.add(LocalDate.of(1910, 6, 22)); // K. Zuse

        System.out.println("Iterating over elements . . .");
        for (LocalDate date : pq)
            System.out.println(date);
        System.out.println("Removing elements . . .");
        while (!pq.isEmpty())
            System.out.println(pq.remove());
    }
}
```

We can use `LocalDate` as elements of the priority queue as the `LocalDate` class implements the `Comparable<LocalDate>` interface

Iteration over the elements of a `PriorityQueue<LocalDate>` does not ensure that dates will be ordered chronologically during the traversal. Indeed, if you run this program, dates will be printed in a seemingly arbitrary order

However, if we extract the dates one by one using the `remove` method, we will get them chronologically ordered. That is because `remove` always returns the earliest date among those currently available in the priority queue



<Exercises>

Which of the following statements about Abstract Data Types (ADTs) is correct?

- A. A linked list is an example of an ADT
- B. An ADT is defined in terms of its structure
- C. A, B, and D are incorrect
- D. A set is an example of an ADT



<Exercises>

Which of the following statements about ADTs in Java are correct?

- A. Java uses parameterized abstract classes to express the concept of an ADT
- B. Java does not provide any ADT implementations
- C. Java uses generic interfaces to express the concept of an ADT
- D. Java does not provide any parametrized abstract class to help implementors of interfaces to develop their own classes



<Exercises>

Which of the following statements about the Iterator interface in Java is correct?

- A. One can add elements to the collection being traversed with the add method
- B. Calling the next method with the iterator located at the end of the collection returns the last element in the collection
- C. They can be used to walk through the elements of Lists, but not those of Queues
- D. They expose the underlying data storage layout of the collection being traversed
- E. The hasNext method can always be called no matter the position of the iterator
- F. The next method can always be called no matter the position of the iteration



<Exercises>

If you need to frequently add/remove elements in the middle of a collection, which collection class implementation would you use?

- A. ArrayDeque
- B. ArrayList
- C. LinkedList
- D. PriorityQueue
- E. Array



<Exercises>

If you need efficient random access to the elements of a collection using their index, which collection class implementation would you use?

- A. ArrayDeque
- B. ArrayList
- C. LinkedList
- D. PriorityQueue
- E. Array



<Exercises>

Assume that the current position of a `ListIterator` is “A B | C D E F”. Which would be the position the iterator and list state after `ListIterator.remove()`?

- A. “A | C D E F”
- B. “A B | D E F”
- C. Cannot tell without knowing the full state of the `ListIterator`
- D. “A B | C D E F”
- E. Run-time exception



<Exercises>

Which collection implementation would you use if you want to be able the efficiently extract the smallest element of the collection?

- A. ArrayDeque
- B. ArrayList
- C. LinkedList
- D. PriorityQueue
- E. Array



Hash Tables

- > Linked lists and arrays let us specify the order in which we want to arrange the elements. However, if we are looking for a particular element, and we don't know its position, we need to visit all elements until we find a match. That can be costly if the collection contains many elements
- > If one does not care about ordering, there are data structures that let us find the elements much faster
- > A well-known data structure for finding objects quickly is the **hash table**. A hash table computes an integer, known as **hash code**, for each object
- > A **hash code** is typically derived from the instance fields of an object, ideally in such a way that objects with different data yield different codes

The table shows the hash codes generated by the `hashCode` method as implemented by the `String` class

String value	Hash Code
"Lee"	72,268
"lee"	107,020
"eel"	100,300

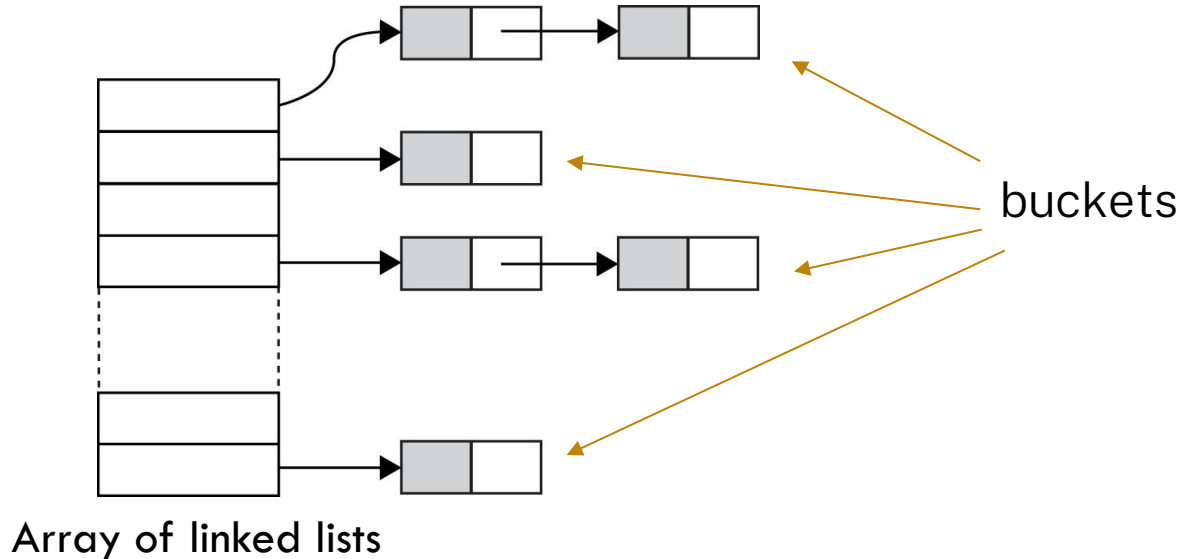


Hash Tables

> Recall that if we define our own classes, then we are responsible for implementing our own hashCode method. Also, **the implementation of hashCode must be compatible with that of the equals method:**

if `a.equals(b)`, then `a.hashCode()==b.hashCode()`

> In Java, hash tables are implemented as an array of linked lists, called **buckets**



Hash Tables

- > To find the place of an object in the table, the hash table computes the hash code of the object, and then reduces the hash code modulo the total number of buckets (i.e., the size of the array)
- > The resulting number is the index of the bucket that may hold the element
- > For example, if an object has hash code 72268 and there are 128 buckets, then the object is placed in bucket number 76, as the remainder of the division of 72268 and 128 is 76 ($72268 \% 128 = 76$)
- > Assuming insertion, if the bucket is empty, then we just insert the element into the bucket
- > However, sometimes, the hash table may hit a non-empty bucket. This is called **hash collision**
- > In the case of a collision, the hash table needs to search for the element into the bucket to determine whether such element is already present in the table
- > If the hash code of the objects in the table are reasonably randomly distributed and the number of buckets is large enough, only a few comparisons per insertion will be needed
- > If the hash table gets too full, it needs to be **rehashed**. To rehash the table, a table with more buckets is created, all elements are inserted into the new table, and the original table is discarded
- > In Java, there is a parameter of hash tables known as **load factor**. E.g., if load factor is 0.75 (default value), and the table is more than 75% full, it is automatically rehashed with twice as many buckets



HashSet<E> class implementation of Set<E> interface

- > Hash tables can be used to implement several important data structures
- > The simplest among these is the Set ADT, represented with the Set<E> generic interface in Java
- > A set is a collection of elements without duplicates. The add method of a set tries to find the element to be added, and adds it if it is not yet present
- > The Collections Framework supplies a HashSet<E> class that implements a set based on a hash table
- >> We add elements to the HashSet with the add method
- >> The contains method makes a fast look-up to see if an element is already in the set, i.e., it only checks the elements in a single bucket, and NOT all elements in the collection
- >> The hash set iterator visits all buckets in turn. Since hashing scatters all elements around in the table, they are visited in a seemingly random order
- > We can control the initial size of the HashSet (which defaults to 16) and the load factor (which defaults to 0.75) using the appropriate constructors of the HashSet
- >> **IMPORTANT NOTE:** HashSet admits mutable objects. Thus, if you mutate an object after inserting it into a HashSet, then the object is no longer in the right position. This can lead to a variety of undesired effects and BUGs in computer programs. It is thus a highly discouraged practice



TreeSet<E> class implementation of NavigableSet<E> interface

- > Tree sets are similar to hash sets, with one added improvement: a tree set is a **sorted** collection
- > Elements are inserted into the tree set in any order. However, when we iterate over this collection, the elements are automatically presented in sorted order
- > For example, if we execute the following Java code:

```
Set<String> sorter = new TreeSet<String>();  
sorter.add("Bob");  
sorter.add("Amy");  
sorter.add("Carl");  
for (String s : sorter) System.out.println(s);
```

then the strings in the set are printed in alphabetical order: Amy, Bob, Carl

- > This is achieved by a data structure called **red-black tree**, which gives the name to the set. Red-black trees work in such a way that whenever an element in the tree is added, it is placed in the proper sorting position. Thus, the iterator always visits the elements in the proper sorted order
- > Adding an element to a red-black tree is slower than adding it to a hash table. But it is still much faster than adding it to an array or linked-list



TreeSet<E> class implementation of NavigableSet<E> interface

> Just as with priority queues, a tree set must hold objects implementing the Comparable<E> interface. Alternatively, one may supply an object implementing the Comparator<E> interface to the constructor of the tree set (as shown in the example below):

```
class TreeSetDemo {  
    public static void main(String [] args) {  
        Set<Item> parts = new TreeSet<Item>();  
        parts.add(new Item("Toaster", 1234)); parts.add(new Item("Widget", 4562)); parts.add(new Item("Router", 9912));  
        System.out.println(parts);  
        Set<Item> sortByDescription = new TreeSet<Item>(Comparator.comparing(Item::description));  
        sortByDescription.addAll(parts);  
        System.out.println(sortByDescription);  
    }  
}
```

```
record Item(String description, int partNumber) implements Comparable<Item> {  
    public int compareTo(Item other) {  
        return Integer.compare(partNumber, other.partNumber);  
    }  
}
```

This call returns an object that implements the `Comparator<Item>` interface. Such comparator object will be used by the tree set to internally sort the items alphabetically by their description

Item is an inner record definition that implements the `Comparable<Item>` interface. Thus, we can use Item as elements of a tree set



java.util.HashSet<E> class (Section 9.3, Core Java Vol I book)

Method Signature	Description
HashSet()	constructs an empty hash set.
HashSet(Collection<? extends E> elements)	constructs a hash set and adds all elements from a collection.
HashSet(int initialCapacity) HashSet(int initialCapacity, float loadFactor)	construct an empty hash set with the specified capacity and load factor, or a load factor of 0.75. If the ratio size/capacity exceeds the load factor, the hash table will be rehashed into a larger one.
static <E> HashSet<E> newHashSet(int numMappings)	constructs an empty hash set with sufficient initial capacity to hold the expected number of elements (numMappings) without rehashing.



java.util.TreeSet<E> class (Section 9.3, Core Java Vol I book)

Method Signature	Description
TreeSet()	
TreeSet(Comparator<? super E> comparator)	construct an empty tree set.
TreeSet(Collection<? extends E> elements)	
TreeSet(SortedSet<E> s)	construct a tree set and add all elements from a collection or sorted set (in the latter case, using the same ordering).

java.util.SortedSet<E> interface (Section 9.3 Core Java Vol I book)

Method Signature	Description
Comparator<? super E> comparator()	returns the comparator used for sorting the elements, or null if the elements are compared with the compareTo method of the Comparable
E first(); E last()	return the smallest or largest element in the sorted set.



java.util.NavigableSet<E> interface (Section 9.3, Core Java Vol I book)

Method Signature	Description
E higher(E value); E lower(E value)	return the least element >value or the largest element <value, or null if there is no such element
E ceiling(E value); E floor(E value)	return the least element ≥value or the largest element ≤value, or null if there is no such element
E pollFirst(); E pollLast()	returns the index of the element that would be returned by the next remove and return the smallest or largest element in this set, or null if the set is empty

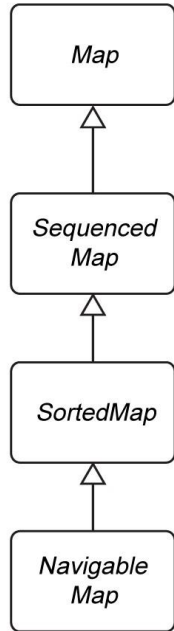


The concept of mapping and the Map ADT

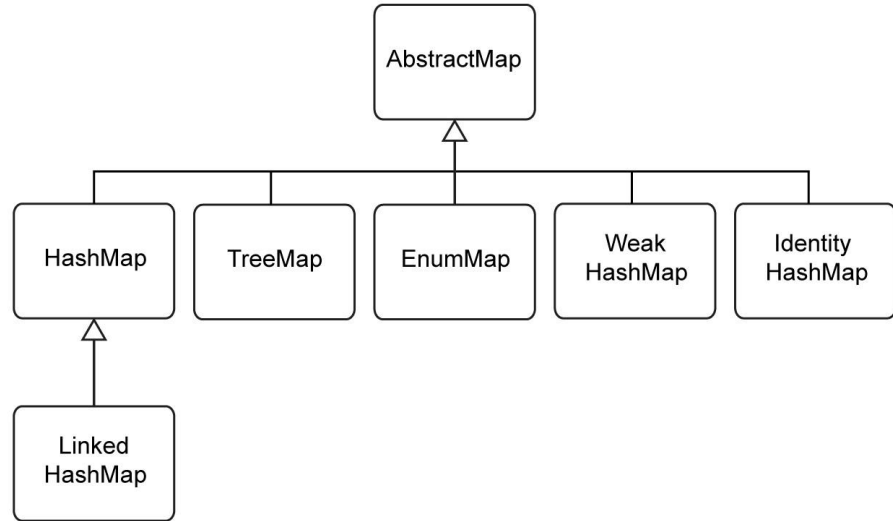
- > A **mapping** can be defined as a relation between inputs and outputs that associates a unique (i.e., one and only one) output to each input
- > (Many) examples of mappings:
 - >> A continuous mathematical function $y = f(x)$ (mapping among x-values and y-values)
 - >> A look-up index (e.g., mapping between words and page numbers in a book, between names and phone numbers in a contact list, etc.)
 - >> Even a list might be interpreted as a mapping between consecutive integers (i.e., indices starting from 0) and list elements
- > A **Map** is a special mapping where the correspondence among inputs and output is explicitly stored
- > Inputs are called **keys**, and outputs are called **values**
- > Each key is associated to one and only one value (recall that maps are mappings)
- > If a given key is associated to a value, we refer to these two as **key-value pair**, and say that maps store key-value pairs
- > Maps are also known as dictionaries, associative arrays, or symbol tables in other programming languages



Map<K,V> interfaces hierarchy, and Map<K,V> class implementations



Interfaces



Classes



Map<K,V> class implementations

Collection Type	Description
HashMap	A data structure that stores key/value associations using hash tables
TreeMap	A map in which the keys are sorted using red-back trees
EnumMap	A map in which the keys belong to an enumerated type
LinkedHashMap	A map that remembers the order in which entries were added
WeakHashMap	A map with values that can be reclaimed by the garbage collector if they are not used elsewhere
IdentityHashMap	A map with keys that are compared by ==, not equals



Basic Map Operations

> The Java Collections Framework provides two general-purpose implementations for map: (1) `HashMap<K,V>` and (2) `TreeMap<K,V>`. Both classes implement the `Map<K,V>` interface

> A hash map hashes the keys, and a tree map uses an ordering on the keys to organize them in a search tree. The values are **NEVER** hashed nor ordered/compared

> As with sets, hashing is usually a bit faster, and it is the preferred choice if we don't need to visit the keys in sorted order

> The code below creates a hash map for storing objects of the `Employee` class, and adds a new key-value pair to the map using the `put` method of the `Map<K,V>` interface:

```
% Map <String, Employee> staff = new HashMap<String, Employee>();  
% var harry = new Employee("Harry Hacker");  
% staff.put("987-98-9996", harry); // first argument is the key, and second the value
```

> In order to extract an object out of the map, we must supply the key to the `get` method (*and therefore remember the key*):

```
% var key_employee_id= "987-98-9996";  
% Employee e = staff.get(key_employee_id); // gets harry, out of the key associated to harry
```



Basic Map Operations

> If there is no value associated to the key provided to `get`, `get` returns `null`

> Sometimes we may have a good default value that can be used for keys that are not present in the map. To this end, we can use the `getOrDefault` method:

```
% Map<String, Integer> scores = . . .;  
% int score = scores.getOrDefault(studentUid, 0); // gets 0 if studentUid is not present
```

> If you call twice the `put` method with the same key, the second value replaces the first one (Note that a key cannot be associated to two different values). Indeed, *put* returns the previous value associated with its key argument (or null otherwise)

> The `remove` method removes an element with a given key from the map. The `size` method returns the number of key-value pairs currently stored in the map

> The easiest way of iterating over the keys and values of a map is the `forEach` method. We supply a lambda expression that receives two arguments, the key and the value. That expression is invoked for every map entry, entry by entry:

```
% scores.forEach((k, v) ->System.out.println("key=" + k + ", value=" + v));
```

> Let us illustrate all these methods in action with an example (next slide)



Basic Map Operations Example

```
import java.util.Map;
import java.util.HashMap;
import java.time.LocalDate;
class MapDemo {
    public static void main(String [] args) {
        Map<String, LocalDate> birthdays = new HashMap<String, LocalDate>();
        birthdays.put("Grace Hopper", LocalDate.of(1906, 12, 9));
        birthdays.put("Ada Lovelace", LocalDate.of(1915, 12, 10));
        birthdays.put("John von Neumann", LocalDate.of(1903, 12, 28));
        birthdays.put("Konrad Zuse", LocalDate.of(1910, 6, 22));
        System.out.println(birthdays); // prints all entries
        birthdays.remove("Konrad Zuse"); // remove a key-value pair
        birthdays.put("Ada Lovelace", LocalDate.of(1815, 12, 10)); // replace the value associate to a key
        System.out.println(birthdays.get("Ada Lovelace")); // look up a value given its key
        birthdays.forEach((k, v) -> System.out.println("key=" + k + ", value=" + v)); // iterate through all key-value pairs
    }
}
```

TO-THINK: if we change HashMap by TreeMap: (1) would the code compile? (2) if yes, would the code necessarily generate on screen the key-value pairs in the same order?

TO-THINK: which value does the put method return in all these calls?

TO-THINK: which value does the put method return in this particular case?



Updating Map entries

> A tricky part of dealing with maps is updating an entry. Normally, we get the old value associated with a key, update it, and put back the updated value

> However, we need to handle **the special case of the first occurrence of a key**

> Consider, for example, using a Map to count how often words occur in a sentence (i.e., a String). We could do something like the following within a loop:

```
% counts.put(word, counts.get(word) + 1)
```

> However, that does not work properly the first time a word is encountered. In such a case, `get` returns `null`, and a `NullPointerExceptionOccurs` (why?)

> A simple remedy is to use the `getOrDefault` method:

```
% counts.put(word, counts.getOrDefault(word, 0) + 1);
```

> Another approach is to first call the `putIfAbsent` method. It only puts a value if the key was previously absent (or mapped to `null`):

```
% counts.putIfAbsent(word, 0);
```

```
% counts.put(word, counts.get(word) + 1); // now we know that get will succeed
```



Updating Map entries

> A much better alternative is to use the `merge` method, which simplifies this common operation:

```
% counts.merge(word, 1, Integer::sum); // TO-THINK: how would you do this with a lambda?
```

> The `merge` operation associates the value 1 with the word if it was not previously present, and otherwise combines the previous value and 1, using the `Integer::sum` function

> Now consider another common situation. We want to associate a `Set` with each key. That is, the value associated to a key in the `Map` is in turn a `Set`. For example, in a book's index, each term has a set of page numbers where the term occurs

> This piece of code cleverly updates the `Set` associated to a given term:

```
% Map<String,TreeSet<Integer>> bookIndex = new TreeMap<String, TreeSet<Integer>>();  
% bookIndex.computeIfAbsent(term, _ -> new TreeSet<>()).add(pageNumber);
```

> If the term is present in the index, then `computeIfAbsent` just returns the `Set` associated to the term. If not, then it calls the lambda expression with its first argument (i.e., `term`), and returns the output of the lambda expression. Thus, in either case, we can safely call the `add` method of a `Set`

> **TO-THINK:** In the example of the book index, why one may consider a `TreeMap` and `TreeSet` for the book index and the set of pages associated to a set, respectively?



Map views

- > Maps do NOT implement the `Iterable` interface. Thus, we cannot extract an iterator out of a `Map`
- > However, we can extract **views** of the map, that is, objects of classes that implement the `Collection` interface (or one of its sub-interfaces)

> In particular, there are three views we can extract:

- >> The Set of keys
- >> The Collection of values, which is not a set (**why not?**)
- >> And the Set of key-value pairs

TO-THINK: why the first and last views are sets?

> We can extract these views with the following three methods of the `Map` interface, respectively:

```
Set<K> keySet();  
Collection<V> values();  
Set<Map.Entry<K, V>> entrySet();
```

where the elements of the `Set` returned by `entrySet` are objects of a class implementing the `Map.Entry` interface



Map views

> For example, we can enumerate all keys of a Map using the following code:

```
Set<String> keys = map.keySet();  
for (String key : keys)  
    do something with key
```

> If we want to look at both keys and values simultaneously, we can iterate over the elements of the view returned by the `entrySet()` method:

```
for (Map.Entry<String, Integer> entry : counts.entrySet()) {  
    String k = entry.getKey(); Integer v = entry.getValue();  
    do something with k, v  
}
```

> The `Map.Entry` instances are connected to the Map. For example, we can use an Entry object to update a value in the underlying Map during the loop:

```
for (Map.Entry<String, Integer> entry : counts.entrySet()) {  
    String k = entry.getKey();  
    Integer v = entry.getValue();  
    entry.setValue(v + 1); // same as counts.put(k, v + 1);  
}
```



Map views (to investigate)

(1) As we have seen, the `keySet()` method of a `Map` returns an object of a class that implements the `Set` interface. How does the program behave if we add a key to this `Set` using the `add` method?

(2) We can extract an iterator out of the object returned by `keySet()` (**why?**). How does the program behave if we call the `remove()` method of the `Iterator` interface using this iterator (when we have walked over at least one element of the `Set`)?



java.util.Map<K,V> interface (Section 9.4, Core Java Vol I book)

Method Signature	Description
<code>V get(Object key)</code>	gets the value associated with the key; returns the object associated with the key, or <code>null</code> if the key is not found in the map. Implementing classes may forbid <code>null</code> keys.
<code>default V getOrDefault(Object key, V defaultValue)</code>	gets the value associated with the key; returns the object associated with the key, or <code>defaultValue</code> if the key is not found in the map.
<code>V put(K key, V value)</code>	puts the association of a key and a value into the map. If the key is already present, the new object replaces the old one previously associated with the key. This method returns the old value of the key, or <code>null</code> if the key was not previously present. Implementing classes may forbid <code>null</code> keys or values.
<code>void putAll(Map<? extends K, ? extends V> entries)</code>	adds all entries from the specified map to this map.
<code>boolean containsKey(Object key)</code>	returns <code>true</code> if the key is present in the map.
<code>boolean containsValue(Object value)</code>	returns <code>true</code> if the value is present in the map.
<code>default void forEach(BiConsumer<? super K, ? super V> action)</code>	applies the action to all key/value pairs of this map.



java.util.Map<K,V> interface (Section 9.4, Core Java Vol I book)

Method Signature	Description
<code>default V merge(K key, V value, BiFunction<? super V,? super V,? extends V> remappingFunction)</code>	If key is associated with a non-null value v, applies the function to v and value and either associates key with the result or, if the result is null, removes the key. Otherwise, associates key with value. Returns <code>get(key)</code> .
<code>default V compute(K key, BiFunction<? super K,? super V,? extends V> remappingFunction)</code>	Applies the function to key and <code>get(key)</code> . Either associates key with the result or, if the result is null, removes the key. Returns <code>get(key)</code> .
<code>default V computeIfPresent(K key, BiFunction<? super K,? super V,? extends V> remappingFunction)</code>	If key is associated with a non-null value v, applies the function to key and v and either associates key with the result or, if the result is null, removes the key. Returns <code>get(key)</code> .
<code>default V computeIfAbsent(K key, Function<? super K,? extends V> mappingFunction)</code>	Applies the function to key unless key is associated with a non-null value. Either associates key with the result or, if the result is null, removes the key. Returns <code>get(key)</code> .
<code>default void replaceAll(BiFunction<? super K,? super V,? extends V> function)</code>	Calls the function on all entries. Associates keys with non-null results and removes keys with null results.
<code>default V putIfAbsent(K key, V value)</code>	If key is absent or associated with null, associates it with value and returns null. Otherwise returns the associated value.



java.util.Map<K,V> interface (Section 9.4, Core Java Vol I book)

Method Signature	Description
<code>Set<Map.Entry<K, V>> entrySet()</code>	returns a set view of Map.Entry objects, the key/value pairs in the map. You can remove elements from this set and they are removed from the map, but you cannot add any elements.
<code>Set<K> keySet()</code>	returns a set view of all keys in the map. You can remove elements from this set and the keys and associated values are removed from the map, but you cannot add any elements.
<code>Collection<V> values()</code>	returns a collection view of all values in the map. You can remove elements from this collection and the removed value and its key are removed from the map, but you cannot add any elements.



java.util.HashMap<K,V> class (Section 9.4 Core Java Vol I book)

Method Signature	Description
HashMap()	
HashMap(int initialCapacity)	
HashMap(int initialCapacity, float loadFactor)	construct an empty hash map with the specified capacity and load factor (a number between 0.0 and 1.0 that determines at what percentage of fullness the hash table will be rehashed into a larger one). The default load factor is 0.75.
static <K, V> HashMap<K, V> newHashMap(int numMappings)	constructs an empty hash map with sufficient initial capacity to hold the expected number of entries (numMappings) without rehashing.



java.util.TreeMap<K,V> class (Section 9.4, Core Java Vol I book)

Method Signature	Description
TreeMap()	constructs an empty tree map for keys that implement the Comparable interface.
TreeMap(Comparator<? super K> c)	constructs a tree map and uses the specified comparator for sorting its keys.
TreeMap(Map<? extends K, ? extends V> entries)	constructs a tree map and adds all entries from a map.
TreeMap(SortedMap<? extends K, ? extends V> entries)	constructs a tree map, adds all entries from a sorted map, and uses the same element comparator as the given sorted map.



java.util.SortedMap<K,V> interface (Section 9.4, Core Java Vol I book)

Method Signature	Description
Comparator<? super K> comparator()	returns the comparator used for sorting the keys, or null if the keys are compared with the compareTo method of the Comparable interface.
K firstKey(); K lastKey()	return the smallest or largest key in the map.



Why generic algorithms?

- > In addition to implementing collection classes, the Java Collections Framework also provides several very useful **generic** algorithms
- > Generic collection interfaces have a major advantage: *if we use them wisely, we only need to implement our algorithms once*
- > Let us illustrate the concept with an example: computing the maximum element in a collection
- > Traditionally, programmers would implement such an algorithm with a loop. Three different (efficient) variants of this loop for an array, ArrayList, and LinkedList could look like:

```
if (a.length == 0)
    throw new NoSuchElementException();
T largest = a[0];
for (int i = 1; i < a.length; i++)
    if (largest.compareTo(a[i]) < 0)
        largest = a[i];
```

array

```
if (v.size() == 0)
    throw new NoSuchElementException();
T largest = v.get(0);
for (int i = 1; i < v.size(); i++)
    if (largest.compareTo(v.get(i)) < 0)
        largest = v.get(i);
```

ArrayList

```
if (l.isEmpty())
    throw new NoSuchElementException();
Iterator<T> iter = l.iterator();
T largest = iter.next();
while (iter.hasNext()) {
    T next = iter.next();
    if (largest.compareTo(next) < 0)
        largest = next;
}
```

LinkedList



Why generic algorithms?

> These loops are **tedious to write** and **error-prone**. For each loop, we need to double check: Is there an off-by-one error? Do the loops work correctly for empty containers? What happens with containers with only one element?

> We don't want to test and debug this code every time nor to implement a whole zoo of methods for each particular data structure, such as these:

```
static <T extends Comparable> T max(ArrayList<T> v);  
static <T extends Comparable> T max(LinkedList<T> l);
```

> The Collection interfaces provide **a powerful solution to this problem**: it is a much better practice to think about which is the more general Collection interface for which we can implement the operation efficiently, and then implement the methods in terms of this interface

> We have seen in the previous slide that this operation does not actually require random access. Thus, we can implement the max method using the Collection interface (i.e., the most general possible interface)



Why generic algorithms?

> The method below implements the `max` operator such that it takes an object of any class implementing the `Collection` interface:

```
public static <T extends Comparable> T max(Collection<T> elements) {  
    if (elements.isEmpty()) throw new NoSuchElementException();  
    Iterator<T> iter = elements.iterator();  
    T largest = iter.next();  
    while (iter.hasNext()) {  
        T next = iter.next();  
        if (largest.compareTo(next) < 0)  
            largest = next;  
    }  
    return largest;  
}
```

> Since arrays are **NOT** collections, we would need to write a separate method for arrays

> The `Collections` and `Arrays` **classes** provide the basics: sorting, binary search, and some simple utility algorithms



Sorting

> The `sort` method in the `Collections` class sorts a collection that implements the `List` interface:

```
% List<String> staff = new ArrayList<String>();  
% // fill collection  
% Collections.sort(staff);
```

> The method assumes that the list elements implement the `Comparable` interface

> If we want to sort the list in a different way, we can pass a `Comparator` as a second argument. For example, the following code sorts a list of employees by increasing salary:

```
% Collections.sort(staff, Comparator.comparingDouble(Employee::getSalary));
```

> If we want to sort the list in descending order, we can use the static method `Comparator.reverseOrder()`:

```
% Collections.sort(staff, Comparator.reverseOrder());
```

where the elements will be ordered in reverse order with respect to the ordering given by the `compareTo` method of the element type

> Similarly we can sort by descending salary using the following code:

```
% Collections.sort(staff, Comparator.comparingDouble(Employee::getSalary).reversed());
```



Binary Search

- > To find an object in an array, we normally visit all elements until we find a match ([linear search](#))
- > However, if the array is sorted, we can look at the middle element and check whether it is larger than the element that we are trying to find. If so, keep looking in the first half of the array; otherwise, look in the second half. That cuts the problem in half in each step. For example, if the array has 1024 elements, we will locate the match (or confirm that there is none) after 10 steps (in the worst case scenario)
- > The `binarySearch` static method of the `Collections` class implements this algorithm:

```
int Collections.binarySearch(elements, target);
```

where `elements` must implement the `List<E>` interface, and `E` the `Comparable` interface. **`elements` MUST be sorted for `binarySearch` to actually work as expected**
- > The method returns the index of the element within `elements` for which `element.compareTo(target)` returns 0 (*if there is any*). If there is no such element, then it returns a negative number that we can use to determine the index at which `target` can be introduced in `elements` to keep the collection sorted. In particular, if `i` is the output, then the insertion index is `-i-1`.
- > To be actually efficient, ***the input List must support efficient random access*** (i.e. to implement the `RandomAccess` interface)



Simple Algorithms

> The `Collections` class contains several simple but useful generic algorithms, such, as, e.g., finding the maximum element of a collection

> Other methods include copying elements from one list to another, filling a container with a constant value, or reversing a list

> While certainly we may implement this simple algorithms our own, using a unified set of generic algorithms can greatly help other programmers (and ourselves in the future) reading our code

> For example, look at this loop (can you quickly tell what it does?):

```
for (int i = 0; i < words.size(); i++)  
    if (words.get(i).equals("C++")) words.set(i, "Java");
```

> Now consider the variant of this code that calls the `replaceAll` method in the `Collections` class:

```
words.replaceAll(words, "C++", "Java");
```

> The table in the next slides introduces the simple algorithms in the `Collections` class. We encourage you to test these methods your own to get familiarized with them and check your understanding



Converting between Collections and Arrays

> If we have an array that we need to turn into a collection, the `List.of` method serves this purpose. For example:

```
% String[] names = . . . ;  
% List<String> staff = List.of(names);
```

> Obtaining an array out of a collection it's a bit trickier. The recommended way since Java 11 is to use the `toArray` default method from the `Collections` interface, as follows:

```
% String[] values = staff.toArray(String[]::new);
```

Recall from the lambdas lecture, that `String[]::new` is a method reference with semantics equivalent to the lambda expression `n -> new String[n]`



java.util.Collections class (Section 9.6, Core Java Vol I book)

Method Signature	Description
<pre>static <T extends Comparable<? super T>> int binarySearch(List<T> elements, T key)</pre>	
<pre>static <T> int binarySearch(List<T> elements, T key, Comparator<? super T> c)</pre>	search for a key in a sorted list, using a binary search if the element type implements the RandomAccess interface, and a linear search in all other cases. The methods are guaranteed to run in $O(a(n) \log n)$ time, where n is the length of the list and $a(n)$ is the average time to access an element. The methods return either the index of the key in the list, or a negative value i if the key is not present in the list. In that case, the key should be inserted at index $-i - 1$ for the list to stay sorted.



java.util.Collections class (Section 9.6, Core Java Vol I book)

Method Signature	Description
<code>static <T extends Comparable<? super T>> T min(Collection<T> elements)</code>	
<code>static <T extends Comparable<? super T>> T max(Collection<T> elements)</code>	
<code>static <T> min(Collection<T> elements, Comparator<? super T> c)</code>	
<code>static <T> max(Collection<T> elements, Comparator<? super T> c)</code>	return the smallest or largest element in the collection. (The parameter bounds are simplified for clarity.)
<code>static <T> void copy(List<? super T> to, List<T> from)</code>	copies all elements from a source list to the same positions in the target list. The target list must be at least as long as the source list.
<code>static <T> void fill(List<? super T> l, T value)</code>	sets all positions of a list to the same value.



java.util.Collections class (Section 9.6, Core Java Vol I book)

Method Signature	Description
<code>static <T> boolean addAll(Collection<? super T> c, T... values)</code>	adds all values to the given collection and returns true if the collection changed as a result.
<code>static <T> boolean replaceAll(List<T> l, T oldValue, T newValue)</code> ^{1.4}	replaces all elements equal to <code>oldValue</code> with <code>newValue</code> .
<code>static int indexOfSubList(List<?> l, List<?> s)</code>	
<code>static int lastIndexOfSubList(List<?> l, List<?> s)</code>	return the index of the first or last sublist of <code>l</code> equaling <code>s</code> , or <code>-1</code> if no sublist of <code>l</code> equals <code>s</code> . For example, if <code>l</code> is <code>[s, t, a, r]</code> and <code>s</code> is <code>[t, a, r]</code> , then both methods return the index <code>1</code> .
<code>static void swap(List<?> l, int i, int j)</code>	swaps the elements at the given offsets.
<code>static void reverse(List<?> l)</code>	reverses the order of the elements in a list. For example, reversing the list <code>[t, a, r]</code> yields the list <code>[r, a, t]</code> . This method runs in $O(n)$ time, where n is the length of the list.



java.util.Collections class (Section 9.6, Core Java Vol I book)

Method Signature	Description
<code>static void rotate(List<?> l, int d)</code>	rotates the elements in the list, moving the entry with index <code>i</code> to position <code>(i + d) % l.size()</code> . For example, rotating the list <code>[t, a, r]</code> by 2 yields the list <code>[a, r, t]</code> . This method runs in $O(n)$ time, where n is the length of the list.
<code>static int frequency(Collection<?> c, Object o)</code>	returns the count of elements in <code>c</code> that equal the object <code>o</code> .
<code>boolean disjoint(Collection<?> c1, Collection<?> c2)</code>	returns <code>true</code> if the collections have no elements in common.



Thank you



Australian
National
University

TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY)
CRICOS PROVIDER CODE: 00120C